로봇 운영 매뉴얼

역할 지정 · RViz · 경기장 전환
로봇 추가 · 4대4 모의 경기 · B-Human 이식 내역

현재 성공 기준

robo04 / ROS 2 Humble / practice 3.9×2.6m

실제 이동·공 탐색 접근·RViz 표시: 사용자 관찰상 성공

실측 위치 정확도·다중 로봇 팀 경기: 다음 단계에서 확인

작성 기준 2026년 11월 9일

빠른 실행 안내 — 현재 robo04

설치가 끝난 현재 구성 기준입니다. 아래 순서를 따르고 실행 창을 유지합니다.

1 터미널 1: 기본 기능 전체

```
bash /home/robo04/incoming/real_robot_20261109/base.sh
```

2 실제 시작 위치에 세운 뒤 물리 START로 기립

우리 골라인 왼쪽 두 T 사이, 중앙보다 왼쪽 약 76cm, 상대 골대 중앙 방향.

3 터미널 3: START 이후 초기 위치 한 번 입력

```
bash /home/robo04/incoming/real_robot_20261109/init_pose.sh
```

4 터미널 2: 보행·킥·경기 행동 실행

```
bash /home/robo04/incoming/real_robot_20261109/start.sh
```

5 터미널 3: 원하는 역할 하나 선택

```
bash /home/robo04/incoming/real_robot_20261109/role.sh attacker --seconds 10
bash /home/robo04/incoming/real_robot_20261109/role.sh defender --seconds 10
bash /home/robo04/incoming/real_robot_20261109/role.sh goalkeeper --seconds 15
```

6 터미널 4: X11 전달을 켠 새 SSH 연결에서 RViz

```
bash /home/robo04/incoming/hsl_view_20261109/start.sh
```

정지 명령 — 터미널 3

```
bash /home/robo04/incoming/real_robot_20261109/role.sh stop
```

역할 명령은 세 개를 연속 실행하지 말고 원하는 것 하나를 고릅니다. 이미 이동했다면 시작 좌표를 다시 입력하지 않습니다.

완전 종료: 역할 stop → 로봇 지지 → 물리 STOP으로 토크 OFF → 실행 창 Ctrl+C.

목차

빠른 실행 안내 — 현재 robo04

1. 현재 사용 중인 구성

2. 재부팅 후 켜는 순서

3. 시작할 때 역할 부여·실행·전환

4. RViz 켜기와 위치 확인

5. 연습장 ↔ 경기장 지도 전환

6. 로봇을 추가할 때의 신원 설정

7. 4대4 모의 경기 구성

8. 종료와 흔한 혼동

9. B-Human에서 가져온 기능과 근거

10. 로봇 8대 설정 기록란

1. 현재 사용 중인 구성

기준 소스 : `MX3_Match_Sim/MX3_Match_Sim/src/mx3_src + src/hsl_src`

로봇 접속 : `robo04@192.168.1.161`

로봇 작업 폴더 : `/home/robo04/sim2real_ws`

기본/역할 실행 묶음 : `/home/robo04/incoming/real_robot_20261109`

RViz 관찰 묶음 : `/home/robo04/incoming/hsl_view_20261109`

경기장 지도 : `practice`, 외곽 3.9×2.6m

선수 번호 : 4번, 현재 단독 역할 시험

보행 정책 : `hsl_bringup/models/walk_v0_6_13500.onnx`

킥 정책 : `hsl_bringup/models/walkkick_v1_5b_22300.onnx`

ROS 환경 : `ROS_DOMAIN_ID=42`, `ROS_LOCALHOST_ONLY=1`

이 문서의 일상 실행 명령은 설치가 끝난 현재 robo04 기준이다. 사용자는 실제 이동·공을 찾으러 가는 동작과 RViz 표시가 잘 된다고 보고했다. 실물 킥 성공률, 실측 자기위치 오차, 여러 대의 팀 통신과 4대4 경기까지 검증한 상태는 아니다. 기존 정책의 재학습이나 센서 검증 전체 반복을 재개 조건으로 두지 않는다.

2. 재부팅 후 켜는 순서

매번 전송·빌드할 필요는 없다. MobaXterm으로 로봇에 접속한 터미널을 3개 준비한다. RViz를 볼 경우 X11을 켠 관찰용 터미널을 하나 더 연다.

터미널 1

실행 내용 : 하드웨어·카메라·YOLO·오도메트리·공 추정·자기위치

창 유지 여부 : 유지

터미널 2

실행 내용 : 실제 보행·킥 정책과 경기 행동

창 유지 여부 : 유지

터미널 3

실행 내용 : 초기 위치, 역할 시작·전환·정지

창 유지 여부 : 명령 입력용

터미널 4, 선택

실행 내용 : RViz 관찰

창 유지 여부 : 유지

2.1 터미널 1: 기본 기능 전체

```
bash /home/robo04/incoming/real_robot_20261109/base.sh
```

기존 개별 하드웨어·카메라·YOLO·월드모델 실행 명령을 중복해서 실행하지 않는다. 카메라와 검출기가 올라오는 동안 초기 TF 대기 경고는 일시적으로 나타날 수 있다. 계속되는 오류와 초기 잠깐의 대기를 구분한다.

2.2 로봇 배치와 물리 START

현재 유지하기로 한 시작 위치:

- 우리 골대에서 상대 골대를 바라볼 때, 우리 골라인 왼쪽 두 T마커 사이.
- 몸통 중심의 바닥 투영점은 골라인 위, 골라인 중앙에서 왼쪽 약 76cm.
- 몸통 정면은 상대 골대 중앙 방향.
- 근사 초기 좌표 `x=-1.95, y=+0.7583333, yaw=0`.

기존 방식대로 지지한 상태에서 물리 START 버튼으로 기립시킨다. START가 자기위치를 초기화하므로 초기 위치 입력은 그 이후에 한다. 이 위치를 바꿔야 하는 시험에서는 새 위치·방향과 이유를 먼저 안내하고 초기 좌표도 함께 바꾼다.

2.3 터미널 3: 초기 위치 입력

실제로 위 시작 위치에 세웠을 때 한 번 입력한다.

```
bash /home/robo04/incoming/real_robot_20261109/init_pose.sh
```

로봇이 이미 이동했는데 RViz를 켜거나 역할을 바꾼다는 이유로 이 명령을 다시 실행하지 않는다. 다시 초기화가 필요하면 현재 실제 배치에 해당하는 좌표를 입력한다.

2.4 터미널 2: 실제 제어 실행

```
bash /home/robo04/incoming/real_robot_20261109/start.sh
```

실제 관절 목표가 모터에 연결된다. 초기 경기 상태는 INITIAL, 보행 모드는 stop이다. 다음 절의 역할 명령으로 역할을 선택하고 행동을 시작한다. 현재 `start.sh`는 launch 인자를 전달하는 스크립트가 아니다. `start.sh assigned_role:=goalkeeper` 같은 명령 대신 아래 `role.sh`를 사용한다.

3. 시작할 때 역할 부여·실행·전환

현재 robo04 한 대에서 아래 명령 중 원하는 역할 하나를 실행한다. `role.sh`가 기존 행동을 정지시키고, 역할을 지정하고, 보행/킥과 PLAYING을 활성화한다.

공격

명령의 역할 이름 : `attacker`

현재 구현된 행동 : 자신을 공 담당으로 지정, 탐색 → 공 뒤 접근 → 정렬 → 조건 충족 시 킥

수비

명령의 역할 이름 : `defender`

현재 구현된 행동 : 우리 골대와 공 사이의 수비 위치로 이동, 공 방향을 바라봄

골키퍼

명령의 역할 이름 : `goalkeeper`

현재 구현된 행동 : 골대 앞에서 공 위치에 따라 좌우 위치 조정, 우리 페널티박스 근처 공을 직접 처리

수비 명령은 현재 코드에서 수비 위치 선정 역할이다. 고정 수비 역할이 상황에 따라 자동 공격수로 바뀌는 구성은 아니다. 골키퍼도 위치 선정과 가까운 공 처리 구현이며, 별도의 다이빙·팔을 이용한 세이브 정책이 들어 있다는 뜻은 아니다.

짧게 관찰하고 자동 정지

```
bash /home/robo04/incoming/real_robot_20261109/role.sh attacker --seconds 10
```

```
bash /home/robo04/incoming/real_robot_20261109/role.sh defender --seconds 10
```

```
bash /home/robo04/incoming/real_robot_20261109/role.sh goalkeeper --seconds 15
```

정지 명령까지 계속 수행

시간 옵션을 생략하면 명령 실행 프로그램이 종료된 뒤에도 로봇의 해당 역할은 계속 활성화된다.

```
bash /home/robo04/incoming/real_robot_20261109/role.sh attacker
```

역할 변경은 다른 역할 명령을 실행하면 된다. 현재 프로그램이 먼저 정지를 요청한다.

```
bash /home/robo04/incoming/real_robot_20261109/role.sh defender
```

정지와 상태 조회:

```
bash /home/robo04/incoming/real_robot_20261109/role.sh stop
bash /home/robo04/incoming/real_robot_20261109/role.sh status
```

역할을 바꿀 때마다 시작 위치로 돌아가거나 초기 좌표를 다시 입력할 필요는 없다. 역할 정지/시간 종료는 기립 토크를 유지한다.

4. RViz 켜기와 위치 확인

4.1 MobaXterm 설정

기존 제어 창은 유지한다. 관찰용 SSH 세션은 Advanced SSH settings의 X11-forwarding을 켜고 새로 접속한다. MobaXterm의 X server도 실행 중이어야 한다. ROS 노드와 RViz는 로봇에서 돌고 창만 Windows로 전달된다. 일반 TCP Tunneling 목록에 ROS 포트를 추가하거나 `ROS_LOCALHOST_ONLY`를 바꿀 필요는 없다.
MobaXterm 공식 문서

터미널 4에서 확인:

```
echo "$DISPLAY"
```

`localhost:10.0`처럼 값이 있으면 된다. 번호는 세션마다 다르다. DISPLAY가 비어 있으면 해당 관찰 세션의 X11 설정을 확인한다. DISPLAY를 임의로 덮어쓰지 않는다.

4.2 터미널 4: 관찰 실행

```
bash /home/robo04/incoming/hsl_view_20261109/start.sh
```

현재 로봇에서 잘 열렸던 명령이다. 소스 교체나 빌드는 필요 없다. 이 관찰 프로그램은 로봇 위치와 공 토픽을 읽어 시각화 토픽만 발행한다. 제어와 초기 위치를 바꾸지 않는다.

흰 선 : 현재 HSL 지도

노란 점 : 지도상 골포스트

파란 화살표 : 추정한 로봇 위치와 몸통 방향

청록 선 : 관찰 이후 추정 위치 궤적, 위치 보정 점프도 포함

주황 공 : 지도상 공 추정, 마지막 관측 경과 시간 표시

상단 x/y/yaw : 위치[m], 방향[도]

quality/validity : 추정기 내부 품질 지표, 실측 정확도와는 구분

landmarks/cycle : 해당 추정 주기에 사용된 랜드마크 수

receive_age / STALE : 수신 경과 시간 / 로봇 위치 수신 중단

지도 원점은 경기장 중앙, +X는 상대 골대 방향, +Y는 우리 골대에서 상대를 볼 때 왼쪽이다. 실제 로봇의 몸통 중심 바닥 투영점과 지도 위치를 대조한다. 정지한 상태에서 라인·마커까지의 거리를 재고, 이동 중 같은 라인을 지날 때 화면에서도 지나는지 확인한다. 화면이 매끄럽거나 quality=2여도 실측 좌표가 맞는지 별도로 대조한다.

새 로봇에 관찰 파일이 없을 때만 PC의 `deploy/hsl_view_20261109.tar.gz`와 SHA 파일을 전송하고, `sha256sum -c` 후 `/home/<사용자>/incoming`에 푼다. robo04 성공본에 다시 설치할 필요는 없다.

5. 연습장 ↔ 경기장 지도 전환

5.1 이 프로젝트의 지도 이름

`practice`

- **외곽 길이×폭 :** 3.9×2.6m
- **우리/상대 골라인 X :** -1.95 / +1.95m

`s_field`

- **외곽 길이×폭 :** 9×6m
- **우리/상대 골라인 X :** -4.5 / +4.5m

여기서 `s_field`는 현재 프로젝트에 들어 있는 큰 경기장 지도다. 실제로 사용할 모의 경기장의 실측 규격이 이 값과 맞는지 확인한다. 프로파일은 인식·전략이 해석하는 지도의 크기이고, 실행 중인 기체가 실물인지 시뮬레이터인지를 바꾸는 옵션은 아니다.

현재 성공본의 기본 실행/역할 실행과 RViz는 `practice`로 고정돼 있다. 현재 스크립트에 `field_profile:=s_field`를 덧붙이는 것만으로 전환되지 않는다. `ros2 param set ... field_profile ...`만으로도 전환하지 않는다. 추정기와 행동은 시작 시 지도를 생성하므로 해당 코드 경로와 시작 설정을 맞춘 뒤 재시작한다.

5.2 현재 성공본에서 바꿀 파일

먼저 역할 stop → 로봇 지지 → 물리 STOP으로 토크 OFF → 터미널 2와 1, 관찰 창 종료 순서로 정리한다. 아래 파일들을 복사해 보관한 뒤 변경한다. 이 문서는 전환 방법이며 현재 연습장 설정을 자동 변경하지 않는다.

sim2real_ws/src/hsl_src/hsl_bringup/launch/real_base.launch.py : 자기위치 노드의 `'field_profile': 'practice'` → `'field_profile': 's_field'`

sim2real_ws/src/hsl_src/hsl_bringup/launch/real_role_trial.launch.py : 행동 노드의 동일 항목을 `'s_field'`로

incoming/hsl_view_20261109/view.launch.py : `'field_profile:=practice'` → `'field_profile:=s_field'`

incoming/hsl_view_20261109/start.sh : 검사 문자열 `'String value is: practice'`를 `'String value is: s_field'`로, 안내 문구의 예상값도 변경

incoming/real_robot_20261109/init_pose.sh : 새 실제 시작 위치에 맞는 x/y/방향으로 변경

표의 상대 경로는 `/home/robo04` 기준이다. `nano` 등으로 해당 항목을 수정한다. 되돌릴 때는 위 프로파일 값을 모두 `practice`로 돌리고 시작 위치도 연습장 실제 배치에 맞춘다.

launch 파일 설치 반영은 hsl_bringup만 빌드한다:

```
source /opt/ros/humble/setup.bash
source /home/robo04/sim2real_ws/install/setup.bash
cd /home/robo04/sim2real_ws
colcon build --base-paths src --packages-select hsl_bringup \
  --symlink-install --executor sequential
```

이후 2절 순서로 다시 실행한다. 시작 위치는 별도로 새 배치를 먼저 결정한다. 예를 들어 큰 경기장 우리 골라인 중앙에 실제로 세우기로 했다면 `x=-4.5, y=0, yaw=0`이 된다. 이는 위치를 바꾸는 예시이며, 지금 유지하기로 한 연습장 시작 위치를 변경하라는 지시는 아니다.

새 터미널에서 세 곳의 실제 프로파일을 확인한다:

```
source /opt/ros/humble/setup.bash
source /home/robo04/sim2real_ws/install/setup.bash
export ROS_DOMAIN_ID=42
export ROS_LOCALHOST_ONLY=1
ros2 param get /hsl_self_localizer field_profile
ros2 param get /hsl_behavior field_profile
ros2 param get /hsl_estimate_view field_profile
```

마지막 명령은 RViz 관찰 프로그램도 실행 중일 때 사용한다. 세 곳이 같은 지도를 사용해야 한다. 설정 수정 후에는 초기 배포 tar.gz를 다시 적용해 변경 설정을 덮어쓰지 않는다. 배포기는 예상 소스 해시가 다르면 거부할 수 있으므로 새 설정은 별도 기록/백업으로 유지한다.

5.3 외곽 크기 외에 확인할 내부 치수

현재 HSL 지도와 예전 `humanoid_localization` 시각화는 practice 내부 치수가 다르다. RViz 관찰 묶음은 현재 HSL 지도를 사용한다.

외곽 길이×폭

- **현재 HSL practice :** 3.9×2.6
- **예전 practice 시각화 :** 3.9×2.6
- **HSL s_field :** 9×6

센터서클 반지름

- **현재 HSL practice :** 0.325
- **예전 practice 시각화 :** 0.35
- **HSL s_field :** 0.75

페널티박스 폭

- **현재 HSL practice :** 약 1.733
- **예전 practice 시각화 :** 2.17
- **HSL s_field :** 4.0

골대 폭

- **현재 HSL practice :** 0.78
- **예전 practice 시각화 :** 1.13
- **HSL s_field :** 1.8

단위는 m다. 어떤 지도 값이 실제 바닥·골대와 맞는지는 실측으로 정한다. 외곽만 맞고 내부 마커가 다르면 자기위치 정확도에 영향을 줄 수 있다. 규격을 새로 맞출 때는 C++ `hsl_world_model/include/hsl_world_model/field_model.hpp`와 Python `hsl_behavior/hsl_behavior/geometry.py`를 함께 맞추고 두 패키지를 빌드해야 한다. RViz는 Python Field를 읽는다. 표시만 실제처럼 바꾸고 추정 지도는 그대로 두지 않는다.

6. 로봇을 추가할 때의 신원 설정

현재 성공한 `real_robot_20261109`는 robo04 전용이다. 다른 로봇에 그대로 실행하면 홈 경로·USB 장치·보정값·선수 번호가 맞지 않는다. 새 로봇을 연결할 때 다음 항목을 정한다.

SSH 사용자/IP : 기체마다 고유 IP, 실제 로그인 사용자 기록

홈/작업 폴더 : `/home/<실제 사용자>/sim2real_ws`, 스크립트와 NumPy 경로도 맞춤

STM32/U2D2 장치 : 해당 기체의 `ls -l /dev/serial/by-id/` 결과 사용, robo04의 일련번호를 복사하지 않음

로봇 보정 섹션 : `robot_params.yaml`의 해당 기체 섹션, IMU/trim 및 모터 맵·게인 보존

제어 launch의 robot : `real_role_trial.launch.py`의 현재 `'robot': 'robo04'`를 실제 보정 섹션으로

선수 번호 : 팀 내부에서 1~4가 중복되지 않게 지정

팀 번호 : 같은 팀은 같게, 상대 팀은 다르게 지정

초기 위치 : 기체마다 실제로 세운 위치의 좌표/방향 입력

지도 : 같은 경기 안의 모든 추정기·전략·관찰이 같은 규격 사용

선수 번호는 자기위치, 행동, 팀 통신, GC의 페널티 처리에서 일치해야 한다. 현재 base와 role launch에 들어 있는 `player_number: 4`도 해당 기체 번호로 바꾼다. robo01~04 보정 섹션이 소스에 있다. 새 기체 robo05~08 등의 섹션과 실제 보정은 추가로 필요하다. 선수 번호만 바꾼다고 보정 섹션/USB 포트/홈 경로까지 자동으로 바뀌지는 않는다.

새 기체에서도 기본 기능 → START 기립 → 자기 실제 초기 위치 → 제어 → 짧은 역할 시험 순서로 시작한다. 기존 정책 파일은 유지하며, 새 기체의 포트·보정·모델 로드·위치가 맞는지 필요한 확인만 한다.

7. 4대4 모의 경기 구성

7.1 제안하는 신원표

아래 이름은 배치 계획 예시다. 실제 사용자의 기체 이름/IP로 채운다. 팀 번호 35/36도 프로젝트 예시이며 사용하는 GameController의 팀 번호와 일치시킨다.

robo01

- **팀 번호 :** 35
- **선수 번호 :** 1
- **경기 역할 모드 :** auto, 골키퍼 번호 1

robo02

- **팀 번호 :** 35
- **선수 번호 :** 2
- **경기 역할 모드 :** auto, 필드 선수

robo03

- **팀 번호 :** 35
- **선수 번호 :** 3
- **경기 역할 모드 :** auto, 필드 선수

robo04

- **팀 번호 :** 35
- **선수 번호 :** 4
- **경기 역할 모드 :** auto, 필드 선수

robo05

- **팀 번호 :** 36
- **선수 번호 :** 1

경기 역할 모드 : auto, 골키퍼 번호 1

robo06

팀 번호 : 36

선수 번호 : 2

경기 역할 모드 : auto, 필드 선수

robo07

팀 번호 : 36

선수 번호 : 3

경기 역할 모드 : auto, 필드 선수

robo08

팀 번호 : 36

선수 번호 : 4

경기 역할 모드 : auto, 필드 선수

4대4는 총 8대다. 4대만 있으면 4대0, 2대2 등의 모의 시험으로 진행할 수 있다. 팀 내부 선수 번호가 같으면 서로를 자기 자신으로 판단해 팀 통신에서 제외할 수 있다. 네 대 모두 attacker로 고정하면 모두 공 담당이 되므로 자동 팀 경기와 다르다.

7.2 고정 역할 시험과 자동 팀 경기

현재 role.sh가 지원하는 것은 `attacker/defender/goalkeeper/stop/status`다. `role.sh auto`는 지원하지 않는다. 행동 노드 자체는 `assigned_role=auto`를 지원한다.

자동 모드에서는 팀원 상태와 공 도달 예상 시간을 비교해 공 담당을 선정한다. 나머지 필드 선수는 수비·지원 위치를 배정받는다. 일반적인 4인 구성은 골키퍼 1 + 공 담당 1 + 수비 1 + 지원 1이며, 누가 공 담당인지는 상황에 따라 달라진다. 골키퍼가 공을 직접 처리하면 남은 선수들의 위치 배정도 달라진다. 정책은 같은 보행·킥 정책을 사용하고 상위 전략의 담당 선정 방식이 달라지는 것이다.

선수 번호/팀 통신 설정을 맞춘 뒤, GC 없이 자동 협동만 시험할 때의 기존 기능 조합 예시:

```
# 아래 예시는 robo04 단독시험 환경의 수동 경기 상태를 사용한다.
bash /home/robo04/incoming/real_robot_20261109/role.sh stop
source /opt/ros/humble/setup.bash
source /home/robo04/sim2real_ws/install/setup.bash
export ROS_DOMAIN_ID=42
export ROS_LOCALHOST_ONLY=1
ros2 param set /hsl_behavior assigned_role auto
ros2 param set /actor_onnx_walkkick_v1_node motion_enabled true
ros2 param set /actor_onnx_walking_node gait_mode auto
ros2 param set /hsl_behavior default_game_state playing
```

이는 현재 파라미터 인터페이스로 가능한 절차이며 실제 여러 대 협동 시험은 아직 확인 전이다. GameController 경기에서는 아래 절의 별도 상태 연결을 사용하고 default_game_state는 initial로 유지한다.

7.3 로봇 간 통신

실제 로봇은 각자 자기 컴퓨터에서 실행한다. 현재처럼 `ROS_LOCALHOST_ONLY=1`을 유지하면 다른 기체의 `/joint_goals`, `/cmd_vel`, `/tf`가 ROS로 섞이지 않는다. 분리된 실제 컴퓨터에서는 모두 ROS_DOMAIN_ID=42를 사용해도 이 방식으로 격리할 수 있다. 별도 도메인을 쓸 경우 해당 로봇의 모든 실행/관찰/명령 스크립트를 같은 값으로 바꿔야 한다.

팀 간 데이터 교환은 ROS DDS가 아닌 `hsl_team_comm`의 독립적인 UDP 소켓으로 한다. 따라서 ROS_LOCALHOST_ONLY=1이어도 로봇 간 UDP 팀 통신은 가능하다. 같은 네트워크/서브넷에 연결하고 AP

의 기기 간 격리, 브로드캐스트 전달, 방화벽을 확인한다.

팀 35 : UDP 10035

팀 36 : UDP 10036

GC 경기 상태 수신 : UDP 3838

GC 응답 송신 : 기본 UDP 3939

현재 base/start에는 팀 통신 노드가 포함되지 않는다. 각 로봇에서 별도로 켜거나 경기용 launch에 넣어야 한다. 예를 들어 팀35 선수4번인 robo04의 별도 팀 통신 창:

```
source /opt/ros/humble/setup.bash
source /home/robo04/sim2real_ws/install/setup.bash
export ROS_DOMAIN_ID=42
export ROS_LOCALHOST_ONLY=1
ros2 run hsl_team_comm team_comm --ros-args \
  --params-file /home/robo04/sim2real_ws/install/hsl_bringup/share/hsl_bringup/config/hsl.yaml \
  -p team_number:=35 -p player_number:=4
```

다른 기체는 사용자 경로·팀·선수 번호를 바꾼다. 같은 기체에서 이 노드를 중복 실행하지 않는다. 상태 확인은 해당 기체의 명령 창에서:

```
timeout 10s ros2 topic echo /hsl/team_state hsl_interfaces/msg/TeamState \
  --once --no-daemon
```

팀4대가 켜져 있으면 자기 자신을 제외한 팀원 3명이 최근 상태로 보여야 한다. 상대 팀은 이 목록에 나타나지 않는 것이 현재 통신 필터의 정상 동작이다. 기본 송신 2Hz와 이벤트 송신을 사용한다. 실제 대회 메시지 제한 준수 여부는 별도 운영 규정과 대조한다. 이 패킷은 우리 HSLT 형식이므로 B-Human 원본 팀 통신과 호환된다고 가정하지 않는다.

7.4 GameController 연결 시 반드시 바꿀 부분

현재 real_role_trial.launch.py는 행동의 `/game_state`를 `/hsl/role_trial/game_state`로 우회해 명령으로 PLAYING을 켜는 단독 시험 구성이다. GC만 추가 실행해도 행동과 자동으로 연결되지는 않는다.

경기용 구성을 만들 때 다음을 한 세트로 반영한다:

1. 자기위치·행동·팀 통신·GC의 team/player 설정을 기체별로 일치시킨다.

2. 행동 노드의 위 `/game_state` remapping을 제거해 실제 GC 상태를 받게 한다.

3. assigned_role=auto, goalkeeper_number=1로 시작한다.

4. default_game_state=initial을 유지해 GC가 끊겼을 때 수동 PLAYING으로 돌아가지 않게 한다.

5. game_state_to_lock을 쓰면 행동의 publish_game_stop_lock은 false로 설정하고 시작한다. 이 항목은 생성 시 발행기를 만드는 설정이므로 launch에서 맞추고 재시작한다.

6. 보행 gait_mode=auto와 킥 motion_enabled=true를 준비하고 실제 경기 상태는 GC로 제어한다.

7. INITIAL/READY/SET/PLAYING/FINISHED와 기체별 페널티, GC 수신 중단 시 동작을 확인한다.

8. GC를 사용하는 초기 배치 방식을 정한다. 현재 자기위치 코드는 GC INITIAL 동안 매 주기 walkInPose(사이드라인 진입 위치)로 가설을 고정한다. 따라서 단독 시험의 골라인 초기 좌표를 INITIAL 중 한 번 입력해도 유지되지 않는다. 사이드라인 진입을 사용할 경우 각 기체의 실제 배치와 walk_in_x_inset/walk_in_side/walk_in_outside_margin을 맞춘다. 골라인 출발을 유지할 경우 GC INITIAL 처리도 그 운영에 맞게 수정해야 한다. 이 전환 때는 현재 약속한 시작 위치에서 무엇이 바뀌는지 먼저 안내한다.

이 변경 후 GC 수신 실행 예시(robo04, 별도 환경 설정은 앞 절과 같음):

```
ros2 launch humanoid_game_controller game_controller.launch.py \
  robot:=robo04 team_number:=35 player_number:=4 enable_return:=false
```

enable_return=false는 현재 HSL 추정 토픽과 기존 GC 응답 노드 사이의 어댑터가 아직 없는 상태의 예시다. 기존 GC 응답 노드는 `/localization/vision_pose`와 `/ball/map_point`를 구독하지만 현재 추정은 `/hsl/robot_pose`와 `/hsl/ball`이라는 서로 다른 메시지 타입이다. 이름만 remap해서 연결할 수 없다. 본격 응답을 사용하려면 메시지 변환과 좌표·단위 처리를 연결해야 한다. 현재 GC 수신 구현은 버전20을 기대하므로 사용할 GC의 패킷 버전도 맞춰야 한다.

GC 경기에서는 로컬 role.sh의 수동 PLAYING으로 경기 상태를 덮는 운영을 하지 않는다. 경기 정지는 GC 상태와 각 기체의 정지 수단을 함께 확인한다. 원본 robot.launch.py는 존재하지만 현재 검증된 분리 기동과 모델·환경 설정이 같지 않으므로 기존 base/start와 함께 실행하거나 곧바로 검증된 대체 명령으로 사용하지 않는다.

7.5 양 팀의 지도 방향과 RViz

각 팀은 자기 팀 기준 지도(+X가 공격 방향)를 사용한다. 상대 팀도 자기 골대는 음의 X, 공격 방향은 양의 X다. 공통 관찰 좌표를 A팀 지도와 같게 정하면 B팀의 지도 좌표는 `X=-x_B, Y=-y_B, yaw=wrap(theta_B+π)`로 변환한다. 초기 위치 입력과 팀 공 공유 때 같은 팀끼리 이 기준을 일치시킨다.

현재 관찰기는 한 로봇만 표시한다. 각 로봇에 X11로 접속해 RViz를 하나씩 여는 방식으로 확인할 수 있다. 8대를 한 화면에 합치려면 로봇 식별이 붙은 관찰 데이터 수집과 양 팀의 공통 좌표 변환이 필요하다. 이를 위해 ROS_LOCALHOST_ONLY를 무작정 0으로 바꾸면 절대 이름의 제어 토픽·TF가 섞일 수 있다. 향후 관찰용 전용 수집기를 사용하고 로봇 제어 통신 격리를 유지한다.

7.6 연습장 4대4에서 추가 조정할 항목

지도 프로파일을 줄여도 모든 행동 상수가 자동으로 비례 축소되는 구조는 아니다. 현재 strategy.py의 킥오프 위치에는 x=-2.6m 같은 큰 필드용 값이 있고, 상대 세트플레이 거리 기본값도 1.2m다. practice 골라인은 ±1.95m다. 따라서 GC READY 배치·탐색 위치·회피 반경·수비/지원 간격을 연습장 실측 공간에 맞춰 점검해야 한다. 큰 경기장도 실측 지도와 선수 배치를 먼저 맞춘다.

실물 진행 순서와 확인 기준:

현재

진행 내용 : robo04 단독 역할과 RViz

다음 단계로 넘어갈 근거 : 사용자 성공 보고, 위치 정확도는 실측 비교 필요

다음 1

진행 내용 : 같은 팀 2대, 먼저 정지 상태의 UDP 통신

다음 단계로 넘어갈 근거 : 양쪽 팀원 목록에 상대 1대가 올바른 번호/좌표로 표시

다음 2

진행 내용 : 같은 팀 2대 자동 모드

다음 단계로 넘어갈 근거 : 공 담당 교대와 비담당의 위치 선정, 정지 명령 확인

다음 3

진행 내용 : 같은 팀 4대

다음 단계로 넘어갈 근거 : 팀원 3명 수신, 공 담당·골키퍼·수비·지원 분담 관찰

다음 4

진행 내용 : 2대2 상대 포함

다음 단계로 넘어갈 근거 : 상대 검출/회피/공 다툼과 양 팀 좌표 방향 확인

다음 5

진행 내용 : 4대4 + GC

다음 단계로 넘어갈 근거 : 상태 전환·페널티·재진입·통신 중단·킥 결과를 경기 로그로 확인

기체를 늘릴 때마다 기존 전체 검증을 다시 하는 대신 신원·통신·협동 연결에서 새로 생기는 부분을 확인한다. 시뮬 전용 run_match.sh와 sim_robot.launch.py는 MuJoCo 브리지를 사용하는 별도 경로이며 실물 경기 실행 명령이 아니다.

8. 종료와 흔한 혼동

1. 명령 창에서 `role.sh stop`으로 역할 정지.
2. 로봇을 지지하고 물리 STOP으로 토크 OFF.
3. 제어 창, 기본 실행 창, 관찰 창에서 Ctrl+C.

RViz 종료, 역할 시간 종료, 역할 stop은 토크 OFF와 다르다. 제어 창/기본 창을 다시 켤 때 같은 노드를 중복 실행하지 않는다.

RViz는 되는데 위치가 바닥과 다름 : 현재 좌표·몸 방향, 초기화 시점, 지도 내부 치수

로봇 추가 후 모두 공으로 감 : assigned_role이 모두 attacker인지, auto와 팀 통신 설정

팀원 목록이 비어 있음 : team/player 중복, UDP 포트/브로드캐스트/AP 격리, 실제 팀 통신 노드 실행

번호를 바꿨는데 동작이 예전과 같음 : 실제 launch의 하드코딩된 번호와 실행 중인 파라미터

GC를 켰는데 역할 상태가 안 바뀜 : 단독 시험용 game_state remapping 제거 여부

GC 응답 위치가 0 : HSL 메시지와 기존 응답 노드의 변환 연결 여부

9. B-Human에서 가져온 기능과 근거

검토 기준일: 2026-11-09. 대상 로컬 소스: `C:/Users/jes/Desktop/MX3_Match_Sim/MX3_Match_Sim/src/hsl_src`와 `src/mx3_src`. 공식 대조 기준: BHumanCodeRelease의 고정 커밋 2e3374ba896da8288607f9a1881b876642fe18b8.

9.1. 지금 로봇에서 사용하는 방식

사용자가 기준으로 지정한 MX3_Match_Sim 폴더에는 이미 B-Human 대응 구현인 `hsl_src`가 들어 있었다. 이번 실물 작업은 그 소스와 MX3 하위 계층을 배포하고 실제 센서·정책·역할 제어에 연결한 것이다. 공식 GitHub 전체를 새로 받아 원본 실행 파일을 그대로 실행하는 방식이 아니다.

공식 소스 일부는 대조 목적으로 별도 `sim2real_evidence/bhuman_official_2026`에 보관돼 있다. 이 폴더의 PROVENANCE.json에는 기준 커밋과 선택한 원본 파일의 Git blob SHA가 기록돼 있다. 해당 커밋은 이번 검토의 대조 기준이다. 로컬 이식의 최초 작성자가 정확히 같은 커밋만 사용했는지까지 이 기록으로 소급해 증명되는 것은 아니다.

실행 흐름은 다음과 같다.

```
실제 관절·IMU·카메라/YOLO
          ↓
hsl_world_model : 오도메트리 · 자기위치 · 공/팀 공
          ↓                    ↕ hsl_team_comm
hsl_behavior    : 역할/공 담당 결정 → 접근·정렬·탐색·목 동작
          ↓ cmd_vel / kick_trigger
mx3_src         : 기존 보행·킥 ONNX 정책 → 모터 제어
```

9.2. 수식·코드 구조의 이식이 직접 확인되는 부분

UKF 위치 추정 핵심

UKF 운동 예측

공식 원본 : `Src/Tools/Modeling/UKFPose2D.cpp`

현재 로컬 구현 : `hsl_world_model/include/hsl_world_model/ukf_pose2d.hpp`의 motionUpdate

확인한 대응 : 7개 시그마점, 방향을 반영한 오도메트리 적용, 평균/공분산 계산, 과정·오도메트리 노이즈 추가

시그마점 생성

공식 원본 : 같은 원본의 generateSigmaPoints

현재 로컬 구현 : 같은 로컬 파일의 generateSigmaPoints

확인한 대응 : Cholesky 분해와 양/음 시그마점 구성

랜드마크 갱신

공식 원본 : 같은 원본의 landmarkSensorUpdate

현재 로컬 구현 : 같은 이름의 로컬 함수

확인한 대응 : 상대 랜드마크 예측과 측정 잔차로 위치 보정

선 갱신

공식 원본 : 같은 원본의 lineSensorUpdate와 UKFRobotPoseHypothesis의 선 처리

현재 로컬 구현 : ukf_pose2d.hpp 및 self_localizer_node.cpp

확인한 대응 : 선 방향에 따른 관측 좌표/각도 선택과 갱신 경로

로컬·원본 함수를 직접 대조해 계산 순서와 수식 대응을 확인했다. 로컬은 Eigen 기반 double과 m 단위, 로컬 Pose2D 및 ROS 노드를 쓰므로 파일 바이트가 동일한 복사본은 아니다. 공식 UKFPose2D, 공식 UKFRobotPoseHypothesis.

SelfLocator 구조

현재 `hsl_world_model/src/self_localizer_node.cpp`는 다중 위치 가설을 유지하고, 관절/IMU 기반 오도메트리로 예측한 뒤 관측으로 갱신하며 유효도와 재표본화로 가설을 관리한다. 현재 설정은 12개 가설이다. 최선 가설을 `/hsl/robot_pose`와 TF `map→odom`으로 내보낸다. 이 구조는 원본 SelfLocator의 가설 집합과 운동·관측 갱신 구조에 대응한다. 공식 SelfLocator.

로컬에서는 기존 YOLO의 L/T/X·골포스트 검출과 RealSense 깊이를 사용한다. 랜드마크 대응, 같은 프레임의 랜드마크 쌍으로 만드는 위치 후보, 대칭 가설과 팀 공을 활용하는 보정이 들어 있다. 원본의 인식 입력·프레임워크까지 그대로 가져온 것은 아니다. 추정 오차를 실측으로 검증해야 한다는 점은 이식 여부와 별개다.

선 측정 입력 `/perception/field_lines` 경로도 코드에 있지만, 현재 실물 카메라에서 해당 선분을 공급하는 검출 노드의 동작은 확인되지 않았다. YOLO의 line 클래스 검출만으로 이 선분 입력이 활성화됐다고 보지 않는다.

9.3. 개념을 참고해 단순화하거나 다시 구현한 부분

다음은 로컬 설계 문서·코드와 원본 기능의 대응을 확인한 항목이다. 원본 전체 알고리즘과 같은 동작을 보장하거나 파일을 그대로 복사했다는 의미는 아니다.

팀 전략·역할

B-Human 원본 대응 : StrategyBehaviorControl, Behavior, Tactics

현재 로컬 구현 : `hsl_behavior/hsl_behavior/strategy.py`

실제로 하는 일 / 차이 : 공 담당 선정, 위치 역할과 우선순위, 거리 비용 기반 배정. 원본 Voronoi 전술 구조 대신 규칙식 위치 생성

공 담당 유지

B-Human 원본 대응 : 원본의 active role 구조

현재 로컬 구현 : `choose_active`, `time_to_ball`

실제로 하는 일 / 차이 : 공 도달 예상 시간과 현재 담당 보너스로 담당을 선정

공 접근과 킥

B-Human 원본 대응 : GoToBallAndKick

현재 로컬 구현 : `hsl_behavior/hsl_behavior/skills.py`의 GoToBallAndKick

실제로 하는 일 / 차이 : 멀리 접근 → 공 주변 회전 → 공 뒤 정렬 → 학습 범위 안이면 우리 킥 정책 호출

탐색·목 동작

B-Human 원본 대응 : BallSearch/Head 계열 설계

현재 로컬 구현 : 같은 패키지의 `skills.py`, `head.py`

실제로 하는 일 / 차이 : 회전/이동 탐색, 공 추적, 주기적 랜드마크 훑기. 원본 탐색 파티클/행동 언어 전체 이식은 아님

공 추적

B-Human 원본 대응 : BallStateEstimator

현재 로컬 구현 : `ball_tracker_node.cpp`

실제로 하는 일 / 차이 : 단일 4차원 칼만 상태와 마찰 예측, 이상치 게이트/재초기화. 원본의 전체 가설/인식 처리는 아님

팀 공

B-Human 원본 대응 : TeamBallModelProvider

현재 로컬 구현 : 같은 파일의 팀 공 계산

실제로 하는 일 / 차이 : 자기 관측과 팀원 공 정보를 이용한 군집/가중 융합. 원본 전체 구현과 같지는 않음

팀 메시지

B-Human 원본 대응 : TeamMessageHandler/TeamData

현재 로컬 구현 : `hsl_team_comm/hsl_team_comm/team_comm_node.py`

실제로 하는 일 / 차이 : 위치·공·담당·넘어짐/페널티 등을 자체 UDP 패킷으로 공유. 주기+이벤트 송신

다리 오도메트리

B-Human 원본 대응 : Odometer/보행 오도메트리의 역할

현재 로컬 구현 : `leg_odometry_node.cpp`

실제로 하는 일 / 차이 : 우리 URDF/KDL과 관절·IMU로 직접 구현. 원본 보행엔진을 가져온 것이 아님

공 다툼·회피

B-Human 원본 대응 : 원본 공 처리·킥 선택/상대 고려라는 설계 대응

현재 로컬 구현 : `duel.py`, behavior_node.py

실제로 하는 일 / 차이 : 상대가 가깝다면 가로막기/비껴 차기/대기. 원본 Zweikampf 전체 이식이라고 할 근거는 없음

원본 전략은 Behavior.cpp에서 위치 배정과 역할 배정을 처리한다. 로컬 코드가 언급한 수비 우선·전방·미드필더 우선순위는 t112.cfg의 priorityGroups와 대응한다.

공 접근/킥의 원본 비교 대상은 GoToBallAndKick.cpp다. 현재 로컬은 그 스킬 이름과 접근→킥 연결 개념을 사용하면서 실제 킥 동작은 제공된 ONNX 정책으로 넘긴다.

팀 공 및 통신 원본 비교 대상은 TeamBallModelProvider와 TeamMessageHandler다. 현재 통신은 `HSLT` 헤더의 자체 51바이트 구조체이며 원본 B-Human 패킷을 그대로 사용하지 않는다.

9.4. B-Human에서 가져온 것으로 분류하지 않는 것

보행 ONNX 가중치 : 제공된 MX3/HSL 폴더의 walk_v0_6_13500.onnx

킥 ONNX 가중치 : 제공된 MX3/HSL 폴더의 walkkick_v1_5b_22300.onnx

관측 조립·관절 순서·모터 출력 러너 : mx3_src의 humanoid_policy_runner

모터 통신·버튼·토크/기립 : mx3_src의 humanoid_serial_bridge

RealSense·YOLO : mx3_src의 humanoid_perception

로봇별 IMU·보행 trim/모터 맵 : MX3 로봇 설정

경기장 치수 : 프로젝트의 FieldModel/Field 설정. 실제 바닥과의 일치는 따로 확인

RViz 관찰 노드 : 이번 작업에서 추가한 hsl_view_20261109

CABSL/MODULE/STREAMABLE 프레임워크, SimRobot, 원본 인식 파이프라인·보행/킥 엔진, 휘슬 검출, 패스 대상 선정과 상세 세트플레이, 전체 상대 추적을 실물에 이식했다고 말할 근거는 없다. 골키퍼 위치 선정이 있다는 사실만으로 원본의 모든 골키퍼 동작/다이빙이 들어 있다고 보지 않는다.

9.5. 이번 실물 연결 작업에서 추가·수정한 것

- 제공된 mx3_src+hsl_src를 로봇에 배포하고 빈 디렉터리 누락과 의존성/빌드 문제를 해결했다.

- ball_tracker의 실제 TF 수신 문제를 수정해 현재 v3 경로를 연결했다.
- 고정 공격/수비/골키퍼 선택과 실행 중 역할 변경 인터페이스를 추가했다.
- 전달받은 최종 보행/킥 가중치를 실제 제어 launch에서 명시적으로 선택했다.
- 정지·명령 유효시간과 킥 활성화 제어를 보완하고 실제 역할 제어 스크립트를 만들었다.
- 기본 기능 통합 실행과 위치 관찰용 RViz 구성을 추가했다.

이 항목들은 이번 세션의 실물 통합 수정이며, B-Human 공식 GitHub에서 그대로 내려받은 기능으로 설명하지 않는다. 정답 폴더의 기존 이식과 이번 통합 수정을 구분해서 이력을 남긴다.

9.6. 근거와 검증 범위

hsl_src/DESIGN.md : 작성자가 설명한 이식 대응표와 단순화 결정. 선언만으로 완전한 동일성을 증명하지 않음

로컬 소스의 실제 함수와 토픽 : 지금 들어 있는 동작과 연결 방식

고정 커밋의 공식 함수 대조 : UKF 계산 및 원본 기능/경로의 대응

bhuman_official_2026/PROVENANCE.json : 선택 보관한 원본 파일의 출처와 blob SHA

사용자 실물 관찰 : 실제 이동·공 탐색 접근·RViz 표시의 성공 보고

자기위치 수치 정확도, 실물 킥/골키퍼 성능, 다중 로봇 통신과 경기 완주 여부는 별도 결과가 필요하다. 시뮬 설계 문서에 있는 성능 수치를 이번 실물의 성능으로 옮겨 적지 않는다.

원본 고지 파일은 `hsl_src/LICENSE_BHUMAN.md`에 있고 일부 이식 C++ 파일에도 출처 고지가 있다. 문서/배포에서도 해당 출처를 유지한다. 이 문서는 코드 출처 확인 기록이다.

10. 로봇 8대 설정 기록란

아래는 제안 배치입니다. robo04 이외의 접속 정보는 실제 기체를 연결하며 채웁니다. 팀35/36도 모의 경기 설정과 일치시킵니다.

robo01 · 팀 35 / 선수 1

접속 : 사용자 _____ / IP _____

워크스페이스 : _____

역할/지도 : 경기 계획 auto / practice

ROS : DOMAIN_ID=42 / LOCALHOST_ONLY=1 (각 기체 컴퓨터 내부 ROS 격리)

초기 위치 : 실측 x _____ m / y _____ m / yaw _____ rad

하드웨어 기록 : STM32 by-id _____ / U2D2 by-id _____

robo02 · 팀 35 / 선수 2

접속 : 사용자 _____ / IP _____

워크스페이스 : _____

역할/지도 : 경기 계획 auto / practice

ROS : DOMAIN_ID=42 / LOCALHOST_ONLY=1 (각 기체 컴퓨터 내부 ROS 격리)

초기 위치 : 실측 x _____ m / y _____ m / yaw _____ rad

하드웨어 기록 : STM32 by-id _____ / U2D2 by-id _____

robo03 · 팀 35 / 선수 3

접속 : 사용자 _____ / IP _____

워크스페이스 : _____

역할/지도 : 경기 계획 auto / practice

ROS : DOMAIN_ID=42 / LOCALHOST_ONLY=1 (각 기체 컴퓨터 내부 ROS 격리)

초기 위치 : 실측 x _____ m / y _____ m / yaw _____ rad

하드웨어 기록 : STM32 by-id _____ / U2D2 by-id _____

robo04 · 팀 35 / 선수 4

접속 : robo04@192.168.1.161

워크스페이스 : /home/robo04/sim2real_ws

역할/지도 : 현재 단독 attacker 시험 / practice

ROS : DOMAIN_ID=42 / LOCALHOST_ONLY=1 (각 기체 컴퓨터 내부 ROS 격리)

초기 위치 : 근사 (-1.95, +0.7583333, 0rad)

하드웨어 기록 : STM32 by-id _____ / U2D2 by-id _____

robo05 · 팀 36 / 선수 1

접속 : 사용자 _____ / IP _____

워크스페이스 : _____

역할/지도 : 경기 계획 auto / practice

ROS : DOMAIN_ID=42 / LOCALHOST_ONLY=1 (각 기체 컴퓨터 내부 ROS 격리)

초기 위치 : 실측 x _____ m / y _____ m / yaw _____ rad

하드웨어 기록 : STM32 by-id _____ / U2D2 by-id _____

robo06 · 팀 36 / 선수 2

접속 : 사용자 _____ / IP _____

워크스페이스 : _____

역할/지도 : 경기 계획 auto / practice

ROS : DOMAIN_ID=42 / LOCALHOST_ONLY=1 (각 기체 컴퓨터 내부 ROS 격리)

초기 위치 : 실측 x _____ m / y _____ m / yaw _____ rad

하드웨어 기록 : STM32 by-id _____ / U2D2 by-id _____

robo07 · 팀 36 / 선수 3

접속 : 사용자 _____ / IP _____

워크스페이스 : _____

역할/지도 : 경기 계획 auto / practice

ROS : DOMAIN_ID=42 / LOCALHOST_ONLY=1 (각 기체 컴퓨터 내부 ROS 격리)

초기 위치 : 실측 x _____ m / y _____ m / yaw _____ rad

하드웨어 기록 : STM32 by-id _____ / U2D2 by-id _____

robo08 · 팀 36 / 선수 4

접속 : 사용자 _____ / IP _____

워크스페이스 : _____

역할/지도 : 경기 계획 auto / practice

ROS : DOMAIN_ID=42 / LOCALHOST_ONLY=1 (각 기체 컴퓨터 내부 ROS 격리)

초기 위치 : 실측 x _____ m / y _____ m / yaw _____ rad

하드웨어 기록 : STM32 by-id _____ / U2D2 by-id _____